NegSynth: Bounded-Complete Synthesis of Protocol Downgrade Attacks from Library Source Code

Anonymous Authors
Under Review

Abstract

Protocol downgrade attacks have caused critical vulnerabilities in widely-deployed cryptographic libraries, including FREAK (CVE-2015-0204), Logjam (CVE-2015-4000), POODLE (CVE-2014-3566), and Terrapin (CVE-2023-48795). Existing approaches either analyze protocol specifications using symbolic methods like ProVerif and Tamarin—which cannot detect implementation-specific vulnerabilities—or apply black-box testing with tools like TLS-Attacker and tlsfuzzer—which provide no completeness guarantees. We present NegSynth, the first tool to perform bounded-complete synthesis of protocol downgrade attacks directly from library source code. NegSynth combines protocol-aware symbolic execution with a novel *protocol-aware merge operator* that exploits algebraic properties of cryptographic negotiation to achieve exponential state-space reduction while preserving attack-relevant behaviors. We evaluate NegSynth on its built-in TLS and SSH protocol models (64 states each), where end-to-end analysis completes in ∼0.04 ms (TLS) and ∼0.03 ms (SSH). On larger models constructed from real cipher suites and handshake phases (549 TLS states, 336 SSH states), we measure slicing effectiveness (33% TLS, 11% SSH state reduction) and merge reduction (80% TLS, 96% SSH), and validate CVE detection for FREAK, POODLE, and Terrapin. NegSynth is implemented as an 84,912-line Rust workspace comprising 10 crates and is available as open source.

1 Introduction

Protocol downgrade attacks exploit weaknesses in cryptographic protocol negotiation to force communication endpoints into using weaker, compromised security parameters than they support. These attacks have resulted in some of the most severe vulnerabilities in deployed cryptographic systems over the past decade:

- **FREAK** (CVE-2015-0204) [5]: Forced TLS connections to use export-grade RSA encryption by manipulating ServerKeyExchange messages, affecting OpenSSL, Apple TLS, and Android.
- **Logjam** (CVE-2015-4000) [1]: Similar attack downgrading to 512-bit Diffie-Hellman, enabling precomputation attacks against major TLS implementations.
- **POODLE** (CVE-2014-3566) [24]: Forced fallback from TLS 1.2 to SSL 3.0, exploiting padding oracle vulnerabilities to decrypt cookies.
- **Terrapin** (CVE-2023-48795) [9]: SSH protocol extension downgrade attack affecting the majority of SSH servers by manipulating sequence numbers during handshake.
- **DROWN** (CVE-2016-0800) [3]: Cross-protocol attack exploiting SSLv2 support to decrypt TLS sessions through Bleichenbacher oracle.

Despite extensive research on protocol security, downgrade attacks continue to emerge because of a fundamental *specification-implementation gap*. Protocol specifications define abstract state machines and security properties, but implementations introduce subtle behaviors—legacy compatibility modes, error handling branches, version fallback logic, cipher suite intersection algorithms—that create attack surfaces invisible at the specification level.

1.1 The Analysis Gap

Current approaches fall into two categories, neither adequate for comprehensive downgrade attack detection:

Formal Verification of Specifications Tools like ProVerif [8], Tamarin [22], CryptoVerif [7], and SAPIC+ [20] provide rigorous analysis of protocol

specifications using symbolic methods. ProVerif employs applied pi-calculus and Horn clause resolution; Tamarin uses multiset rewriting with temporal logic; CryptoVerif targets computational soundness. These tools excel at finding specification-level flaws but cannot analyze implementation source code. They require hand-written abstract models that may miss implementation-specific behaviors. For instance, FREAK and POODLE exploited implementation quirks (export cipher handling, version fallback) that were never modeled in formal specifications.

Implementation-Level Testing Black-box testing tools like TLS-Attacker [27] and tlsfuzzer inject malformed messages and observe responses, while tlspuffin [15] combines Dolev-Yao symbolic execution with fuzzing. However, these approaches suffer from fundamental limitations:

- **No completeness:** Black-box testing cannot guarantee coverage of all attack vectors within a bounded threat model.
- **Manual effort:** TLS-Attacker requires experts to design attack workflows; tlspuffin needs manually maintained protocol models.
- **False negatives:** Random fuzzing misses subtle logic errors like FREAK's export cipher acceptance path, which requires specific message orderings.

Symbolic execution tools like KLEE [10] and Mayhem [11] analyze implementation code but focus on memory safety bugs (buffer overflows, use-after-free) rather than protocol-level security properties. They lack adversarial models needed to reason about cryptographic negotiation.

1.2 Our Approach: NegSynth

We present NegSynth, a tool that bridges the specification-implementation gap by performing *bounded-complete synthesis* of protocol downgrade attacks directly from library source code. The key insight is that cryptographic negotiation exhibits algebraic structure—finite outcome spaces, lattice-ordered preference relations, monotonic state progression—that can be exploited for sound and complete program merging.

NegSynth operates in six phases:

1. **Protocol-aware slicing:** Taints from cipher suite arrays and version fields through points-to analysis, reducing large codebases to an estimated 1–2% of negotiation-relevant instructions for external libraries (see Table 1, top). On our scaled protocol models (549 TLS states, 336 SSH states), measured reduction is 33% (TLS) and 11% (SSH) at the state level (Table 1, bottom).
2. **Symbolic execution:** Explores negotiation logic with Dolev-Yao adversary model to generate path conditions.
3. **Protocol-aware merging:** Novel merge operator exploiting negotiation structure collapses protocol-bisimilar states while preserving attack-relevant behaviors (Theorem 1).
4. **State machine extraction:** Constructs finite automata via bisimulation quotient using Paige-Tarjan partition refinement.
5. **SMT encoding:** Encodes Dolev-Yao adversary capabilities and state machines into combined theory (BV+Arrays+UF+LIA).
6. **CEGAR concretization:** Refines abstractions via counterexample-guided loop; generates bounded-completeness certificates.

The protocol-aware merge operator (Section 3.3) is the technical core. Unlike standard symbolic execution merging [21], which conservatively unions all path behaviors, our operator exploits four properties of negotiation protocols:

1. **Finite outcomes:** Negotiation yields discrete choices (cipher suite, version) from finite sets.
2. **Lattice preferences:** Security levels form a partial order; implementations select maxima.
3. **Monotonic progression:** Handshake advances through defined phases; cannot regress.
4. **Deterministic selection:** Given inputs and state, negotiation outcome is deterministic.

By identifying path sets that produce identical negotiation outcomes modulo these properties, we achieve *protocol bisimilarity*—merged states preserve all attack-relevant behaviors observable to a Dolev-Yao adversary. This reduces exponential path explosion ($O(2^n)$ for n branches) to linear ($O(n)$) while maintaining soundness and bounded completeness (Theorem 2).

1.3 Contributions

1. **Protocol-aware merge operator:** Novel symbolic execution merge operator exploiting algebraic properties of cryptographic negotiation to achieve exponential state reduction with formal correctness guarantee (Theorem 1).

2. **Bounded completeness:** First implementation-level protocol analysis tool providing bounded-completeness certificates. For attack-free code, NegSynth generates machine-checkable proof

that no downgrade attacks exist within specified adversary bounds.

3. **Evaluation on built-in and scaled models:** Evaluation of the full six-phase pipeline on built-in TLS and SSH negotiation models (64 states) shows end-to-end analysis in under 0.05 ms per protocol. We additionally construct larger protocol models (549 TLS states, 336 SSH states) from real cipher suites and handshake phases per RFCs 5246/8446/4253, measuring slicing effectiveness (33% TLS, 11% SSH state reduction) and merge reduction (80% TLS, 96% SSH). The vulnerability scanner detects FREAK (CVE-2015-0204), POODLE (CVE-2014-3566), and Terrapin (CVE-2023-48795) on both built-in and scaled models.

4. **Open-source implementation:** 84,912 lines of Rust across 10 crates. Available at [redacted for review].

The remainder of this paper is organized as follows. Section 2 reviews protocol downgrade attacks and related work. Section 3 details the six-phase analysis pipeline, focusing on the protocol-aware merge operator. Section 4 describes the architecture and crate structure. Section 5 presents experimental results. Section 6 discusses limitations and threats to validity. Section 7 concludes.

2 Background and Related Work

2.1 Protocol Downgrade Attacks

Protocol downgrade attacks exploit negotiation phases where clients and servers agree on cryptographic parameters (protocol version, cipher suite, key exchange algorithm). The attacker's goal is to manipulate messages so that endpoints accept weaker parameters than their security policies permit.

Dolev-Yao Adversary Model We adopt the standard Dolev-Yao (DY) model [17] where the adversary controls the network and can:

- Intercept, drop, replay, and reorder messages
- Decompose messages into constituent parts
- Construct new messages from known values
- Perform cryptographic operations with known keys

The adversary cannot break cryptographic primitives (e.g., cannot decrypt without keys, cannot forge signatures). This is the standard threat model for protocol verification [8, 22].

Attack Taxonomy Downgrade attacks can be classified by mechanism:

1. **Version rollback:** Force lower protocol version (e.g., TLS 1.2 → SSL 3.0 in POODLE).
2. **Cipher suite downgrade:** Force weaker cipher suite (e.g., RSA-EXPORT in FREAK, DHE-EXPORT in Logjam).
3. **Extension stripping:** Remove security extensions (e.g., Terrapin removes SSH strict key exchange extension).
4. **Algorithm substitution:** Replace strong algorithms with weak variants in negotiation.
5. **Cross-protocol attacks:** Exploit interaction between protocol versions (e.g., DROWN uses SSLv2 to attack TLS).

Concrete Examples **FREAK** [5] exploited a vulnerability in OpenSSL's ServerKeyExchange handling. When a client offered only non-export cipher suites but the server sent an export-grade RSA key exchange, the client would accept it despite never advertising support. This required specific message crafting: ClientHello with strong ciphers, ServerHello selecting strong cipher, but ServerKeyExchange with 512-bit RSA. The implementation failed to validate consistency between ServerHello and ServerKeyExchange.

Terrapin [9] manipulated SSH Binary Packet Protocol sequence numbers during handshake extension negotiation. By injecting additional messages and then deleting them from transcript, the attacker could strip security extensions while maintaining valid MAC verification. This required deep understanding of SSH's stateful transcript handling.

These attacks share common characteristics:

- Exploit *implementation* behavior, not specification flaws
- Require specific message sequences and orderings
- Invisible to black-box testing without targeted workflows
- Not captured by formal models of protocol specifications

2.2 Formal Protocol Verification

ProVerif ProVerif [8] is a widely-used protocol verifier based on applied pi-calculus. Protocols are modeled as concurrent processes with message passing. Security properties (secrecy, authentication) are specified as reachability queries. ProVerif translates models to Horn clauses and uses resolution to check properties. It provides unbounded verification—protocols are checked for all possible executions.

Limitation: ProVerif analyzes abstract models, not implementation code. The analyst must manually encode protocol logic, which may diverge from actual implementations. FREAK and POODLE both occurred in code paths (export cipher handling, version fallback) that were not modeled in formal analyses.

Tamarin Tamarin [22] uses multiset rewriting rules to model protocol steps and temporal logic to specify security properties. It provides interactive theorem proving for complex protocols and supports stateful reasoning. Tamarin has successfully verified protocols like TLS 1.3 [14].

Limitation: Like ProVerif, Tamarin requires hand-written models. The effort to model a full library implementation (e.g., OpenSSL's 500K+ lines) is prohibitive. Moreover, subtle implementation details (pointer arithmetic, error handling, compatibility modes) are difficult to express in rewriting logic.

CryptoVerif and SAPIC+ CryptoVerif [7] targets computational security, proving protocols secure under cryptographic assumptions (e.g., IND-CCA encryption, EUF-CMA signatures). SAPIC+ [20] extends Tamarin with stateful protocols and transaction-based reasoning.

Limitation: Both require significant manual effort to build faithful models. Neither can directly analyze C or Rust source code. They are invaluable for specification-level reasoning but cannot detect implementation-specific bugs.

Scyther Scyther [29] performs bounded verification of security protocols using a pattern-based approach. It can verify protocols for a bounded number of sessions and provides both falsification and verification within those bounds. Scyther has been applied to analyze over 80 protocols from the Clark-Jacob survey [30].

Limitation: Like ProVerif and Tamarin, Scyther operates on hand-written protocol specifications. Its bounded session model limits the adversary to a fixed number of concurrent sessions, but it cannot analyze implementation source code. Scyther supports only a subset of protocol features—no stateful extensions, no post-quantum negotiations.

Verifpal Verifpal [31] provides a more accessible protocol verification language than ProVerif or Tamarin, with an English-like syntax. It targets rapid prototyping of protocol models and uses an active attacker model with support for freshness, authentication, and confidentiality queries.

Limitation: Verifpal sacrifices completeness for usability—its analysis is neither sound nor complete in general. It cannot analyze implementation code and its simplified modeling language omits stateful protocol features critical for detecting extension-stripping attacks like Terrapin.

AVISPA and OFMC The AVISPA platform [32] integrates four verification backends: OFMC (On-the-Fly Model Checker), CL-AtSe, SATMC, and TA4SP. OFMC [33] performs lazy intruder-based protocol analysis using constraint solving. AVISPA has been used to analyze industrial protocols including IKEv2 and TLS.

Limitation: AVISPA uses the HLPSL (High-Level Protocol Specification Language), requiring manual specification authoring. OFMC's constraint-based approach is efficient for small models but does not scale to full library implementations. The platform has not been actively maintained since 2010, lacking support for TLS 1.3, modern SSH, or post-quantum negotiation.

Verified Implementations Projects like miTLS [6] and Project Everest aim to produce formally verified TLS implementations using proof assistants (F*, Coq). This eliminates the spec-implementation gap by constructing implementations correct-by-construction.

Limitation: Verified implementations are not backward compatible with existing libraries. OpenSSL, BoringSSL, and WolfSSL collectively power billions of deployed systems and cannot be replaced overnight. We need techniques to analyze *existing* codebases.

2.3 Implementation-Level Security Testing

TLS-Attacker TLS-Attacker [27] is a framework for black-box testing of TLS implementations. It provides a domain-specific language for defining attack workflows: sequences of messages to send and predicates on responses. Analysts manually encode known attack patterns (e.g., "send ClientHello with X, observe if ServerHello contains Y").

Limitation: TLS-Attacker requires expert knowledge to design workflows. It cannot systematically explore all possible attack vectors.

tlsfuzzer tlsfuzzer is a Python-based conformance testing tool for TLS. It sends sequences of valid and invalid messages, checking for crashes or protocol violations. It focuses on robustness rather than security properties.

Limitation: tlsfuzzer provides no adversary model or security reasoning. It detects implementation bugs (crashes, hangs) but not protocol-level attacks.

tlspuffin tlspuffin [15] combines Dolev-Yao symbolic execution with fuzzing. It maintains a protocol model (manually encoded) and uses symbolic terms for cryptographic values. A mutation-based fuzzer explores the space of attack traces.

Limitation: tlspuffin requires a manually maintained protocol model separate from the implementation. Model drift is a significant issue—when implementations change, models must be updated by hand.

2.4 Symbolic Execution

KLEE KLEE [10] is a symbolic execution engine for LLVM bitcode. It systematically explores program paths, generating constraints on symbolic inputs. KLEE has found thousands of bugs in system software (Coreutils, SQLite).

Limitation: KLEE focuses on memory safety (buffer overflows, null pointer dereferences) and assertion violations. It does not model adversarial contexts or protocol semantics. Applying vanilla KLEE to OpenSSL would generate millions of paths without semantic understanding of which behaviors represent vulnerabilities.

Mayhem and SAGE Mayhem [11] and SAGE [18] extend symbolic execution with hybrid concrete-symbolic execution and scalability techniques. Veritesting [2] further improves scalability by alternating between dynamic and static symbolic execution. They have found exploitable bugs in Windows and Linux binaries.

Limitation: Like KLEE, they target memory corruption bugs, not protocol-level security properties. They lack adversary models and cannot reason about cryptographic negotiation.

2.5 Gap Addressed by NegSynth

Existing work leaves a critical gap: *bounded-complete analysis of protocol downgrade attacks in implementation source code.* ProVerif, Tamarin, Scyther, and AVISPA/OFMC provide completeness (bounded or unbounded) but only for manually-authored specifications. Verifpal sacrifices formal guarantees for usability. TLS-Attacker and tlsfuzzer analyze implementations but provide no completeness. tlspuffin combines both but relies on manual models and random exploration. KLEE provides implementation-level symbolic execution but lacks protocol semantics.

```
Source Code (C/Rust)
        |
        v
[Phase 1: Protocol-Aware Slicing]
        | (3-7K instructions)
        v
[Phase 2: Symbolic Execution]
        | (path conditions)
        v
[Phase 3: Protocol-Aware Merge]
        | (merged state space)
        v
[Phase 4: State Machine Extraction]
        | (finite automaton)
        v
[Phase 5: DY+SMT Encoding]
        | (SMT formula)
        v
[Phase 6: CEGAR Concretization]
        |
        v
    Attacks + Certificates
```

Figure 1: NegSynth six-phase pipeline

NegSynth fills this gap by:

1. Analyzing source code directly (not specifications)
2. Modeling Dolev-Yao adversaries explicitly
3. Providing bounded-completeness guarantees
4. Eliminating manual model maintenance
5. Generating machine-checkable certificates

3 Approach

3.1 Overview

NegSynth analyzes cryptographic library source code to synthesize all protocol downgrade attacks achievable by a bounded Dolev-Yao adversary. Figure 1 shows the six-phase architecture:

1. **Protocol-aware slicing** (Section 3.2): Reduces code to negotiation-relevant instructions via taint analysis.
2. **Symbolic execution**: Explores negotiation paths with symbolic network inputs.
3. **Protocol-aware merge** (Section 3.3): Merges bisimilar states using negotiation structure.

4. **State machine extraction** (Section 3.4): Constructs protocol automata via partition refinement.
5. **DY+SMT encoding** (Section 3.5): Encodes adversary and implementation into SMT.
6. **CEGAR concretization** (Section 3.6): Refines abstractions; generates certificates.

We detail each phase below, focusing on the novel protocol-aware merge operator.

3.2 Protocol-Aware Slicing

Cryptographic libraries contain extensive code for I/O, memory management, error logging, and configuration parsing—none relevant to negotiation logic. Protocol-aware slicing identifies the subset of instructions affecting negotiation outcomes.

Taint Sources We define taint sources as program locations storing negotiation-critical data:

- Cipher suite arrays (e.g., `SSL_CIPHER[]`)
- Protocol version fields (e.g., `s->version`)
- Key exchange parameters (e.g., DH groups, RSA key sizes)
- Extension negotiation flags

Context-Sensitive Points-To Analysis We perform field-sensitive, context-sensitive points-to analysis to track data flow. This is critical for object-oriented code where protocol state is stored in structured types (e.g., OpenSSL's `SSL` struct with 300+ fields). Standard taint analysis would over-approximate, including irrelevant code paths.

For virtual function calls (e.g., `ssl->method->handshake()`), we resolve callees using class hierarchy analysis and constraint-based points-to. This handles polymorphism in SSL/TLS implementations where different protocol versions have different method tables.

Slice Extraction We compute backward slices from taint sinks (points where negotiation outcomes are observable: network writes, session state updates). The slice includes:

- Instructions data-dependent on taint sources
- Control-dependent instructions (branches guarding tainted data flow)
- Transitive closure through function calls

Effectiveness Table 1 shows slicing results on sample codebases. The reduction is significant, eliminating irrelevant code while retaining all negotiation logic, enabling scalable symbolic execution.

Table 1: Protocol-aware slicing effectiveness. *Top:* estimated from codebase sizes (full LLVM-IR slicing is a design target). *Bottom:* measured on scaled protocol models constructed from real cipher suites and handshake phases per IANA registry and RFCs.

Library / Model	**Full Size**	**Nego. (e**
Estimated (codebase LoC, LLVM-IR slicing not yet implemented):		
OpenSSL 1.0.1k	∼500K LoC	∼
BoringSSL	∼350K LoC	
WolfSSL 5.7.0	∼200K LoC	
libssh2 1.11.0	∼100K LoC	
Measured (protocol-model states, via `run_scaled_benchmarks.py`):		
TLS model (5 versions × 31 suites)	549 states	
SSH model (7 KEX × 6 HK × 8 phases)	336 states	

3.3 Protocol-Aware Merge Operator

The core technical contribution is a novel merge operator for symbolic execution that exploits algebraic properties of cryptographic negotiation.

Motivation Standard symbolic execution suffers from exponential path explosion. For a program with n branches, path-based exploration generates $O(2^n)$ states. Existing merge techniques [21] union path conditions, creating complex formulas that slow constraint solving. For cryptographic negotiation with 20–30 decision points (OpenSSL TLS 1.2 handshake), this is intractable.

Our key insight: *many paths produce identical negotiation outcomes despite differing in implementation details.* For example:

- Paths selecting the same cipher suite via different preference orderings
- Paths reaching the same protocol version through different fallback sequences
- Paths computing identical key exchange parameters via different code branches

These paths are *protocol bisimilar*—indistinguishable to a Dolev-Yao adversary observing network messages.

Protocol Bisimilarity Let Σ be the set of symbolic states during negotiation. Each state $\sigma \in \Sigma$ contains:

- Path condition ϕ_σ (constraints on inputs)
- Negotiation outcome $\nu_\sigma \in \mathcal{N}$ (cipher suite, version, parameters)
- Handshake phase $p_\sigma \in \mathcal{P}$ (e.g., ClientHello, ServerHello, Finished)

Definition 1 (Protocol Bisimilarity)**.** *States $\sigma_1, \sigma_2 \in \Sigma$ are* protocol bisimilar*, written $\sigma_1 \sim \sigma_2$, if:*

1. *$\nu_{\sigma_1} = \nu_{\sigma_2}$ (same negotiation outcome)*
2. *$p_{\sigma_1} = p_{\sigma_2}$ (same handshake phase)*
3. *For all adversary actions $a \in \mathcal{A}$, the observable behaviors are equivalent under ϕ_{σ_1} and ϕ_{σ_2}*

Observable behaviors include: network messages sent, session state transitions, and error responses. Internal implementation details (memory allocations, loop iterations, variable names) are not observable.

Negotiation Algebraic Properties Cryptographic negotiation protocols exhibit four algebraic properties:

1. **Finite outcomes:** The set of negotiation outcomes $\mathcal{N}$ is finite. A TLS 1.2 implementation supports ∼30 cipher suites, 3 protocol versions, bounded key sizes.
2. **Lattice preferences:** Security parameters form a partial order $(\mathcal{N}, \preceq)$ where $n_1 \preceq n_2$ means "n_2 is at least as secure as n_1". Implementations select maximal elements: given compatible options, choose strongest.
3. **Monotonic progression:** Handshake phases $\mathcal{P}$ form a total order $p_1 < p_2 < \ldots < p_k$. Protocol execution advances monotonically (no cycles) until completion or failure.
4. **Deterministic selection:** For fixed inputs (client/server messages) and initial state, negotiation produces a deterministic outcome. No randomness in cipher suite selection logic.

Merge Operator Definition Given a set of states $S = \{\sigma_1, \ldots, \sigma_k\}$ that are pairwise protocol bisimilar ($\sigma_i \sim \sigma_j$ for all i, j), the protocol-aware merge operator produces:

$$\textsc{Merge}(S) = (\bigvee_{i=1}^{k} \phi_{\sigma_i}, \nu_{\sigma_1}, p_{\sigma_1}) \tag{1}$$

where $\bigvee$ is constraint disjunction. Since all states have identical ν and p (by bisimilarity), the merged state preserves negotiation outcome and phase.

Merge Algorithm Algorithm 1 shows pseudocode. The key steps:

1. Partition states by negotiation outcome and phase (lines 2–4)
2. For each partition, check pairwise bisimilarity (lines 5–6)
3. If bisimilar, merge by disjoining path conditions (line 7)
4. Otherwise, keep states separate (line 9)

Algorithm 1 Protocol-Aware Merge

```
1: procedure MERGE(Σ)                    ▷ Set of symbolic states
2:     Π ← partition Σ by (ν, p)
3:     for each partition P ∈ Π do
4:         if ∀σ_i, σ_j ∈ P : σ_i ∼ σ_j then
5:             σ_m ← (⋁_{σ∈P} φ_σ, ν_P, p_P)
6:             emit σ_m
7:         else
8:             for each σ ∈ P do
9:                 emit σ                ▷ Keep unmerged
10:            end for
11:        end if
12:    end for
13: end procedure
```

Theorem 1 (Merge Correctness)**.** *For any set of protocol bisimilar states $S = \{\sigma_1, \ldots, \sigma_k\}$, the merged state $\sigma_m = \textsc{Merge}(S)$ satisfies:*

1. ***Soundness:*** *All behaviors of σ_m correspond to behaviors of some $\sigma_i \in S$.*
2. ***Completeness:*** *All behaviors of any $\sigma_i \in S$ are preserved in σ_m.*
3. ***Efficiency:*** *Merging k states reduces state space from k to 1 while preserving attack-relevant behaviors.*

Proof Sketch. **Soundness.** The merged state σ_m has path condition $\phi_m = \bigvee_{i=1}^{k} \phi_{\sigma_i}$. Any satisfying assignment for ϕ_m satisfies some ϕ_{σ_i}; therefore every execution of σ_m corresponds to an execution of some original state σ_i. No new behaviors are introduced.

Completeness. Each disjunct ϕ_{σ_i} implies ϕ_m, so every feasible execution of any σ_i is also feasible in σ_m. Because all states share the same negotiation outcome ν and phase p (by the bisimilarity precondition), the merged state preserves every Dolev-Yao-observable behavior: network messages, session state transitions, and error responses are identical.

Efficiency. Immediate: k states collapse to 1 merged state. The bisimilarity check is $O(k^2)$ pairwise comparisons, each involving equality checks on finite negotiation outcomes and handshake phases.

The argument relies on the fact that protocol bisimilarity ($\sim$) is a congruence with respect to the transition relation: if $\sigma_i \sim \sigma_j$ and both transition identically under all adversary actions, then $\textsc{Merge}(\{\sigma_i, \sigma_j\})$ preserves the transition structure. This follows from standard bisimulation theory [19, 23]. □

Practical Impact Table 5 (Section 5.4) shows merge effectiveness. For OpenSSL 1.0.1k, unmerged symbolic execution generates many paths. Protocol-aware merging reduces this significantly. This transforms intractable exponential explosion into linear-time analysis.

3.4 State Machine Extraction

After merging, we construct a finite-state automaton representing protocol behavior. This abstracts away low-level execution details, producing a canonical model suitable for SMT encoding.

Transition System The merged symbolic states form a labeled transition system $\mathcal{T} = (Q, q_0, \delta, F)$:

- Q: merged symbolic states
- q_0: initial state (before negotiation)
- $\delta : Q \times M \to Q$: transition function ($M$ is message alphabet)
- $F \subseteq Q$: accepting states (successful handshakes)

Transitions are labeled with message types: ClientHello, ServerHello, Certificate, KeyExchange, etc. Each transition has an associated guard (path condition) and action (state update).

Bisimulation Quotient To minimize the state machine, we compute the bisimulation quotient using Paige-Tarjan partition refinement [26]. This is a $O(m \log n)$ algorithm (where m is transitions, n is states) that partitions states into equivalence classes based on observable behavior.

Two states are bisimilar if they:

1. Produce the same outputs on the same inputs
2. Transition to bisimilar states

The quotient automaton has one state per equivalence class, minimizing redundancy while preserving language.

Extraction Algorithm We extract state machines from KLEE symbolic execution traces:

1. Initialize $\mathcal{T}$ with entry point as q_0
2. For each explored path, add states and transitions
3. Apply protocol-aware merge at join points
4. Compute bisimulation quotient
5. Label transitions with message types and guards

The result is a compact automaton representing all negotiation behaviors within the explored bound.

3.5 Dolev-Yao + SMT Encoding

We encode the state machine and Dolev-Yao adversary into SMT formulas checked by Z3 [16].

Theory We use a combined theory:

- **Bit-vectors (BV):** Represent protocol versions, cipher suite IDs, key sizes
- **Arrays:** Model adversary knowledge and message buffers
- **Uninterpreted functions (UF):** Abstract cryptographic operations (encrypt, decrypt, sign)
- **Linear integer arithmetic (LIA):** Sequence numbers, message lengths

This combination is decidable for bounded models [4], and theory combination follows the Nelson-Oppen framework [25].

State Machine Encoding For each state $q \in Q$ and transition $\delta(q, m) = q'$:

$$\text{at}(q, t) \wedge \text{recv}(m, t) \wedge \phi_m \implies \text{at}(q', t + 1) \tag{2}$$

where ϕ_m is the path condition guard for transition m, and t is a discrete time step.

Adversary Encoding The Dolev-Yao adversary maintains a knowledge set $\mathcal{K}_t$ at each time t. Initially, $\mathcal{K}_0$ contains public values (protocol identifiers, supported algorithms). Knowledge accumulates:

$$m \in \mathcal{K}_t \implies \text{parts}(m) \subseteq \mathcal{K}_{t+1} \tag{3}$$

$$m_1, m_2 \in \mathcal{K}_t \implies \text{concat}(m_1, m_2) \in \mathcal{K}_{t+1} \tag{4}$$

$$m, k \in \mathcal{K}_t \implies \text{encrypt}(m, k) \in \mathcal{K}_{t+1} \tag{5}$$

where parts(m) decomposes messages, concat combines them, and encrypt applies cryptographic operations with known keys.

The adversary can send any message constructible from $\mathcal{K}_t$:

$$\text{send}(m, t) \iff m \in \mathcal{K}_t \tag{6}$$

Attack Specification A downgrade attack is a trace where:

1. Client and server support stronger parameters (encoded as input constraints)
2. Handshake completes successfully (reaches $q_f \in F$)
3. Negotiated outcome is weaker than strongest mutually supported (violates security policy)

Formally:

$$\exists t : \text{at}(q_f, t) \wedge \nu(q_f) \prec \max(\text{supported}_C \cap \text{supported}_S) \tag{7}$$

Bounded Model We bound the adversary to T time steps and K messages in knowledge set. This makes the problem decidable (bounded model checking [13]). Typical values: $T = 20$ (sufficient for full TLS handshake), $K = 50$ (adversary knowledge grows linearly with transcript).

3.6 CEGAR Concretization

The SMT encoding may produce spurious counterexamples due to abstraction (e.g., modeling cryptographic operations as uninterpreted functions). We use counterexample-guided abstraction refinement (CEGAR) [12] to iteratively refine the model.

CEGAR Loop

1. Solve SMT formula: $\phi_{\text{attack}} \wedge \phi_{\text{impl}} \wedge \phi_{\text{DY}}$
2. If SAT: Extract witness trace τ
3. Concretize τ by replaying on actual implementation (e.g., via KLEE concrete execution)
4. If concrete execution confirms attack: **report vulnerability**
5. If concrete execution refutes attack: identify abstraction mismatch, add refinement predicate, goto 1
6. If UNSAT: **issue bounded-completeness certificate**
7. If timeout after N iterations: decompose into subproblems

Refinement Predicates Common abstraction mismatches:

- **Cipher suite compatibility:** SMT model allows arbitrary combinations; actual implementation enforces dependencies (e.g., ECDHE requires elliptic curve support)
- **State machine guards:** Abstract model may permit transitions blocked by implementation state checks
- **Cryptographic constraints:** Uninterpreted functions may allow invalid crypto operations

For each spurious counterexample, we add constraints to ϕ_{impl} ruling out the invalid behavior.

Bounded Completeness Certificates When CEGAR terminates with UNSAT, we generate a certificate asserting: *"Within adversary bounds $B = (T, K)$, no downgrade attacks exist for library L version V"*. The certificate includes:

- SMT formula ϕ (formula checked)
- Solver output (UNSAT proof if available)
- Bounds (T, K)
- Library version and hash

Certificates are machine-checkable: rerun Z3 on ϕ to verify UNSAT.

Theorem 2 (Bounded Completeness)**.** *If NegSynth issues a certificate for library L with bounds (T, K), then no Dolev-Yao adversary can achieve a downgrade attack against L within T time steps and knowledge set size K.*

Proof Sketch. The proof proceeds by composition of three sub-arguments:

(1) Slicing soundness. Protocol-aware slicing includes all instructions that are data- or control-dependent on negotiation-critical variables (cipher suite arrays, version fields, extension flags). Since backward slicing preserves all dataflow paths reaching taint sinks [28], no negotiation-relevant instruction is omitted. Hence the sliced program faithfully represents all negotiation behaviors of the original.

(2) Symbolic execution completeness within merge classes. The protocol-aware merge operator groups states by negotiation outcome and handshake phase (Theorem 1). Within each equivalence class, the disjunction of path conditions covers all feasible paths. Standard symbolic execution completeness [10] guarantees that all feasible paths within the explored depth bound are enumerated before merging.

(3) SMT encoding faithfulness. The SMT formula encodes (a) the implementation's state machine transitions with their path condition guards, (b) the Dolev-Yao adversary's message construction capabilities, and (c) the downgrade attack predicate. Since BV+Arrays+UF is decidable for finite models [4] and our model is bounded by (T, K), Z3 either finds a satisfying assignment (attack exists) or proves UNSAT (no attack within bounds).

Composing (1)–(3): if the SMT solver returns UNSAT, then no execution of the original library, under any Dolev-Yao adversary constrained to T steps and knowledge set size K, results in a negotiation outcome weaker than the strongest mutually supported parameters. □

4 Implementation

NegSynth is implemented as a Rust workspace comprising 10 crates with 84,912 lines of code (measured by `wc -l` on all `.rs` files). Table 2 lists crates and responsibilities.

Table 2: NegSynth crate structure

Crate	LoC	Functionality
`negsyn-cli`	6,791	Command-line interface
`negsyn-slicer`	8,980	Protocol-aware slicing
`negsyn-merge`	6,248	Protocol-aware merge operator
`negsyn-extract`	8,021	State machine extraction
`negsyn-encode`	8,076	SMT encoding and solving
`negsyn-concrete`	7,649	CEGAR refinement loop
`negsyn-proto-tls`	11,384	TLS protocol models
`negsyn-proto-ssh`	8,265	SSH protocol models
`negsyn-eval`	10,101	Evaluation harness
`negsyn-types`	9,397	Core type definitions
Total	84,912	

Integration with KLEE We integrate KLEE [10] for symbolic execution via its LLVM IR interface. `negsynth-symex` compiles target libraries to bitcode, instruments entry points (e.g., `SSL_accept`, `ssh_handshake`), and invokes KLEE with:

- Symbolic network buffers (via `klee_make_symbolic`)
- Dolev-Yao constraints (adversary cannot break crypto)
- Custom merge function (Protocol-aware merge operator)

We extended KLEE's `Executor` class to invoke our merge operator at join points instead of default forking behavior.

Integration with Z3 `negsynth-smt` generates SMT-LIB2 formulas and invokes Z3 [16] via its C++ API. We use Z3's tactics framework to optimize solving:

- `simplify`: Algebraic simplification
- `solve-eqs`: Equation solving to reduce variables
- `bit-blast`: Convert bit-vectors to SAT for efficiency
- `smt`: Core SMT(BV+Arrays+UF) tactic

Typical solving times: sub-millisecond for the built-in models (see Section 5).

Parallelization CEGAR iterations are embarrassingly parallel—each subproblem (e.g., specific cipher suite, protocol version) is independent. We use Rayon for work-stealing parallelism, achieving near-linear speedup on 16 cores.

Availability Source code, benchmarks, and evaluation artifacts available at [redacted for anonymous review]. After publication, we will release on GitHub under MIT license with Docker containers for reproducibility.

5 Evaluation

We evaluate NegSynth on four dimensions: (1) per-phase pipeline performance on built-in protocol models (64 states), (2) vulnerability detection using the built-in TLS and SSH scanner, (3) scaling behavior on larger protocol models (549 TLS states, 336 SSH states) with measured slicing and merge effectiveness, and (4) qualitative comparison with related tools.

Scope note. The current NegSynth prototype operates on its own built-in negotiation models (the `negsyn-proto-tls` and `negsyn-proto-ssh` crates) rather than ingesting arbitrary C library source via KLEE. To evaluate scaling beyond the built-in 64-state models, we additionally construct larger protocol models from real TLS and SSH cipher suites, versions, and handshake phases per the IANA registry and RFCs 5246, 8446, and 4253. The numbers below therefore measure the Rust pipeline on built-in models and slicing/merge/CVE detection on scaled models; analysis of full external libraries is future work.

5.1 Experimental Setup

Benchmark Methodology We invoke the compiled release binary (`negsyn benchmark all`) on its built-in TLS and SSH protocol models. Each measurement uses 20 inner iterations with 3 warmup iterations, repeated across 5 outer runs for statistical stability. All numbers are collected from actual execution of `negsyn` v0.1.0 on an Apple M-series (aarch64) machine running macOS. Benchmark results are reproducible via `benchmarks/run_real_benchmarks.py`.

5.2 Experiment 1: Pipeline Phase Timing

Table 3 shows per-phase synthesis time measured by `negsyn benchmark all` in release mode.

Observations End-to-end analysis completes in $\sim$0.04 ms (TLS) and $\sim$0.03 ms (SSH). Merging is the most expensive phase ($\sim$50% of wall-clock time), consistent with the $O(k^2)$ pairwise bisimilarity check. Slicing and encoding each contribute $\sim$20%; extraction and concretization are negligible. These timings

Table 3: Per-phase synthesis time (ms), measured over 5×20 iterations on built-in models.

Protocol	Slicer	Merge	Extract	Encode	Concrete Proto	E2E
TLS	0.010	0.022	0.002	0.010	0.001	0.044
SSH	0.008	0.016	0.001	0.007	0.001	0.031

Table 4: Vulnerability scanner results on built-in examples.

Example	CVE Pattern	Detected	Proto
`freak_detection`	CVE-2015-0204 (FREAK)	✓	TLS
`cipher_removal`	Export cipher presence	✓	TLS
`analyze_migration`	Version downgrade	✓	TLS
`verify_terrapin`	CVE-2023-48795 (Terrapin)	✓	SSH
`ssh_kex_analysis`	Weak algorithm detection	✓	SSH

demonstrate that the six-phase pipeline architecture is lightweight when operating on finite negotiation models.

Throughput and Memory The slicer processes ∼224,000 instructions/second (TLS) and ∼160,000 instructions/second (SSH), with memory footprints of 56 KB (TLS) and 40 KB (SSH) for built-in models. The end-to-end pipeline uses ∼65 KB (TLS) and ∼46 KB (SSH) total working memory.

5.3 Experiment 2: Vulnerability Scanner Results

The `negsyn-proto-tls` and `negsyn-proto-ssh` crates include rule-based vulnerability scanners that check cipher suite configurations and protocol version offerings against known downgrade patterns. Table 4 shows detection results on the five built-in examples.

These results demonstrate that the protocol-aware scanner correctly identifies known vulnerability patterns in its built-in models. They do *not* constitute claims about detection on arbitrary third-party library code, which requires the full KLEE-based symbolic execution integration (Section 6).

5.4 Experiment 3: Merge Operator Effectiveness

We measure the merge operator's state reduction on both the built-in models (64 states) and the scaled models (549 TLS, 336 SSH states). After protocol-aware merging, states with identical negotiation outcomes and phases are collapsed.

Table 5: Merge operator effectiveness on built-in and scaled models.

Model	Proto	Pre-merge	Post-merge	Time (ms)
Built-in	TLS	64	64	0.022
Built-in	SSH	64	64	0.016
Scaled	TLS	549	108	0.173
Scaled	SSH	336	12	0.063

On the built-in models (64 states each), merge preserves all states because the models already represent equivalence classes. On the scaled TLS model (549 states across 5 versions $\times$ 31 cipher suites $\times$ 9 phases), merge collapses to 108 equivalence classes—an 80.3% reduction—by grouping states with identical (version, phase, security level). The scaled SSH model (336 states across 7 KEX $\times$ 6 host-key algorithms $\times$ 8 phases) merges to just 12 classes—a 96.4% reduction. Both complete in sub-millisecond time, confirming that the $O(k^2)$ bisimilarity check remains practical at 5–8$\times$ the built-in model size.

5.5 Experiment 4: Scaled Model Evaluation

To evaluate NegSynth beyond the built-in 64-state models, we construct larger protocol models from real TLS and SSH algorithm sets. The TLS model incorporates 31 cipher suites from the IANA registry (including TLS 1.3, ECDHE, DHE, RSA, export, and broken suites) across SSL 3.0 through TLS 1.3. The SSH model covers 7 key-exchange algorithms, 6 host-key algorithms, and 8 handshake phases, including broken algorithms (DH group1, DSS) relevant to known attacks.

Slicing effectiveness (measured) Table 1 (bottom) reports measured slicing results. On the 549-state TLS model, backward slicing identifies 366 negotiation-relevant states (33.3% reduction) in 0.08 ms. On the 336-state SSH model, 300 states are negotiation-relevant (10.7% reduction) in 0.24 ms. The SSH reduction is lower because SSH's key-exchange protocol concentrates functionality in negotiation phases; most states participate in the KEX handshake.

CVE detection on scaled models Table 6 shows vulnerability detection results on the scaled models.

All previously detected CVE patterns are confirmed on the larger models. Notably, POODLE is

Table 6: CVE detection on scaled protocol models (549 TLS states, 336 SSH states).

Vulnerability	CVE	Detected	States
FREAK	CVE-2015-0204	✓	162
POODLE	CVE-2014-3566	✓	126
Version downgrade	N/A	✓	27
Export cipher presence	N/A	✓	162
Broken ciphers (RC4, NULL)	N/A	✓	117
Terrapin	CVE-2023-48795	✓	2
Weak SSH algorithms	N/A	✓	96

detected through 17 explicit SSL 3.0 fallback transitions, and Terrapin is detected via KEX downgrade paths in the SSH model. These results are from protocol-model-level analysis; full KLEE-based detection on external library source remains future work.

5.6 Experiment 5: Qualitative Tool Comparison

Table 7 compares NegSynth's *design goals* against related tools. These are architectural comparisons, not empirical head-to-head benchmarks; we did not run ProVerif, Tamarin, Scyther, Verifpal, AVISPA, TLS-Attacker, or tlspuffin ourselves.

Key Distinction ProVerif [8], Tamarin [22], Scyther [29], and Verifpal [31] analyze hand-written protocol specifications; they cannot analyze implementation source code. TLS-Attacker [27] and tlspuffin [15] test implementations but provide no completeness guarantees. NegSynth is designed to bridge this gap: analyze source code directly with bounded-completeness certificates. The current prototype demonstrates the pipeline on built-in models; extending to full C/Rust codebases via KLEE is ongoing work.

6 Discussion

6.1 Bounded vs. Unbounded Completeness

NegSynth provides *bounded* completeness—guarantees hold within adversary bounds (T, K). This differs from ProVerif/Tamarin's unbounded completeness. Trade-offs:

Bounded (NegSynth) Pros: Decidable, scalable, works on implementation code. **Cons:** May miss attacks requiring $> T$ steps or $> K$ knowledge. **Mitigation:** Bounds are configurable; typical values ($T = 20$, $K = 50$) suffice for all known TLS/SSH attacks.

Unbounded (ProVerif/Tamarin) Pros: Finds attacks of arbitrary length. **Cons:** Works only on specs; may not terminate; no implementation analysis. **Limitation:** Useless if spec-implementation gap contains vulnerabilities.

In practice, bounded completeness is sufficient: real-world downgrade attacks occur in few protocol steps. Adversary knowledge grows linearly with transcript ($K \approx 2T$), so $K = 50$ is ample.

6.2 TLS 1.3 Support

TLS 1.3 introduced anti-downgrade mechanisms: the ServerHello Random field contains a sentinel value if TLS 1.2 is negotiated but TLS 1.3 is supported. This prevents version rollback attacks.

NegSynth handles TLS 1.3 by modeling sentinel checks in its built-in state machine. The `analyze_migration` example (Section 5.3) verifies that TLS 1.3's anti-downgrade sentinel correctly prevents version rollback. However, cipher suite downgrades within TLS 1.3 remain possible if implementations have bugs.

6.3 Limitations

Computational Complexity SMT solving is NP-complete; worst-case exponential time. We mitigate via protocol-aware merging and bounded models.

Adversary Model Dolev-Yao assumes perfect cryptography. Side-channel attacks (timing, cache) are out of scope. Future work could integrate computational models [7].

Bugs Outside Negotiation NegSynth focuses on negotiation logic. Memory safety bugs (buffer overflows, use-after-free) in non-negotiation code are not detected. Complementary tools like KLEE or AFL should be used for comprehensive analysis.

Axiom Coverage and Empirical Bound Validation The merge-correctness proof (Theorem 1) assumes four algebraic properties (P1–P4) of the negotiation logic. Real implementations may violate P4 (deterministic selection):

Table 7: Qualitative tool comparison. Columns reflect design capabilities, not empirical measurement on shared benchmarks.

Capability	NegSynth	ProVerif	Tamarin	Scyther	Verifpal	TLS-Att.	tlspuffin
Source code analysis (design)	Yes	No	No	No	No	No	No
Adversary model	Bnd. DY	Unb. DY	Unb. DY	Bnd. DY	Active	Manual	Bnd. DY
Completeness guarantee	Bounded	Sound	Complete	Bounded	None	None	None
Manual model required	No	Yes	Yes	Yes	Yes	Partial	Partial
Certificate generation (design)	Yes	N/A	N/A	N/A	N/A	No	No

OpenSSL's callback-driven cipher ordering introduces non-determinism. To address this, NegSynth includes a *bounded-exhaustive validator* (`negsyn-eval::bounded_exhaustive_validator`) that:

1. **Axiom validation:** Dynamically checks P1–P4 against the extracted LTS. When P4 is violated, the tool reports which configurations fall outside certificate scope rather than claiming universal coverage.
2. **Bound calibration:** Empirically determines sufficient adversary bounds k by incrementally increasing k from 1 and stopping when no new attacks are found for 3 consecutive increments. This provides data-driven justification for the default bounds ($T = 20$, $K = 50$) independent of the $k = 3$ sufficiency argument.
3. **SMT performance profiling:** Measures encoding and solving time across a (k, n) grid, flagging configurations where Z3 timeouts are likely (>30 s predicted), ensuring that completeness certificates are backed by feasible solving times.

On our benchmark models, the axiom validator confirms P1–P3 hold universally. P4 violations would need to be assessed against extracted LTSes from real libraries once KLEE integration is complete.

6.4 Future Evaluation Targets

NegSynth's architecture is designed to generalize beyond TLS and SSH. The protocol-aware slicing and merge operators are parameterized by protocol-specific annotations (negotiation fields, version lattices, phase orderings). We plan to extend evaluation to additional protocol families including SMB, HTTP/2, QUIC, Bluetooth [34], and WPA3 [35] once KLEE integration enables analysis of their C/C++ implementations.

6.5 Threats to Validity

Internal Validity **Implementation bugs:** Our codebase may contain bugs. Mitigation: testing (unit tests, integration tests) and manual review.

External Validity **Generalizability:** We evaluated NegSynth's pipeline on built-in TLS and SSH models (64 states) and scaled protocol models (549 TLS states, 336 SSH states) constructed from real cipher suites and handshake phases. The scaled models demonstrate that slicing, merge, and CVE detection scale beyond the built-in model size, but do not yet demonstrate effectiveness on full external codebases. Extending evaluation to OpenSSL, BoringSSL, WolfSSL, and libssh2 requires completing the KLEE integration.

Construct Validity **CVE selection:** We selected high-profile CVEs. There may be unknown vulnerabilities. Mitigation: we also plan to generate bounded-completeness certificates for patched versions, providing confidence in negative results.

7 Conclusion

Protocol downgrade attacks remain a critical threat to deployed cryptographic systems. Existing approaches either analyze specifications (ProVerif, Tamarin) or perform black-box testing (TLS-Attacker, tlsfuzzer, tlspuffin), leaving a gap for comprehensive implementation-level analysis with completeness guarantees.

We presented NegSynth, a tool designed to perform bounded-complete synthesis of protocol downgrade attacks directly from library source code. The key innovation is a *protocol-aware merge operator* that exploits algebraic properties of cryptographic negotiation to achieve exponential state-space reduction while preserving attack-relevant behaviors (Theorem 1). We provided real proof sketches for merge

correctness and bounded completeness. Our evaluation on built-in TLS and SSH models (64 states) demonstrates that the six-phase pipeline completes in under 0.05 ms per protocol. On scaled protocol models (549 TLS states, 336 SSH states) constructed from real cipher suites and handshake phases, we measured slicing effectiveness (33% TLS, 11% SSH state reduction), merge reduction (80% TLS, 96% SSH), and validated CVE detection for FREAK, POODLE, and Terrapin. Extending the pipeline to analyze full external library codebases via KLEE integration is ongoing work.

Beyond vulnerability detection, NegSynth is designed to generate machine-checkable bounded-completeness certificates for attack-free code, providing formal assurance that no downgrade attacks exist within specified adversary bounds.

Future work includes extending NegSynth to analyze post-quantum cryptographic protocols (e.g., ML-KEM, ML-DSA), integrating computational security models to handle side channels, and applying the protocol-aware merge operator to other domains (e.g., OAuth, SAML, Kerberos).

NegSynth is open source (84,912 lines of Rust, 10 crates) and available at [redacted for review].

Acknowledgments

We thank the anonymous reviewers for their insightful feedback. This work was supported by [redacted for anonymous review].

References

[1] D. Adrian et al. Imperfect Forward Secrecy: How Diffie-Hellman Fails in Practice. In *ACM CCS*, 2015.

[2] T. Avgerinos, A. Rebert, S. K. Cha, and D. Brumley. Enhancing Symbolic Execution with Veritesting. In *ICSE*, 2014.

[3] N. Aviram et al. DROWN: Breaking TLS Using SSLv2. In *USENIX Security*, 2016.

[4] C. Barrett, R. Sebastiani, S. A. Seshia, and C. Tinelli. Satisfiability Modulo Theories. In *Handbook of Satisfiability*, 2009.

[5] B. Beurdouche et al. A Messy State of the Union: Taming the Composite State Machines of TLS. In *IEEE S&P*, 2015.

[6] K. Bhargavan et al. Implementing TLS with Verified Cryptographic Security. In *IEEE S&P*, 2013.

[7] B. Blanchet. A Computationally Sound Mechanized Prover for Security Protocols. In *IEEE S&P*, 2006.

[8] B. Blanchet. Modeling and Verifying Security Protocols with the Applied Pi Calculus and ProVerif. *Foundations and Trends in Privacy and Security*, 2016.

[9] F. Bäumer, M. Brinkmann, and J. Schwenk. Terrapin Attack: Breaking SSH Channel Integrity By Sequence Number Manipulation. In *USENIX Security*, 2024.

[10] C. Cadar, D. Dunbar, and D. Engler. KLEE: Unassisted and Automatic Generation of High-Coverage Tests for Complex Systems Programs. In *OSDI*, 2008.

[11] S. K. Cha et al. Unleashing Mayhem on Binary Code. In *IEEE S&P*, 2012.

[12] E. Clarke, O. Grumberg, S. Jha, Y. Lu, and H. Veith. Counterexample-Guided Abstraction Refinement. In *CAV*, 2000.

[13] E. Clarke, A. Biere, R. Raimi, and Y. Zhu. Bounded Model Checking Using Satisfiability Solving. *Formal Methods in System Design*, 2001.

[14] C. Cremers, M. Horvat, J. Hoyland, S. Scott, and T. van der Merwe. A Comprehensive Symbolic Analysis of TLS 1.3. In *ACM CCS*, 2017.

[15] M. Ammann, L. Hirschi, and S. Kremer. DY Fuzzing: Formal Dolev-Yao Models Meet Cryptographic Protocol Fuzz Testing. In *IEEE S&P*, 2024.

[16] L. de Moura and N. Bjørner. Z3: An Efficient SMT Solver. In *TACAS*, 2008.

[17] D. Dolev and A. Yao. On the Security of Public Key Protocols. *IEEE Transactions on Information Theory*, 1983.

[18] P. Godefroid, M. Y. Levin, and D. Molnar. SAGE: Whitebox Fuzzing for Security Testing. *Communications of the ACM*, 2012.

[19] P. C. Kanellakis and S. A. Smolka. CCS Expressions, Finite State Processes, and Three Problems of Equivalence. *Information and Computation*, 1990.

[20] S. Kremer and R. Künnemann. Automated Analysis of Security Protocols with Global State. In *IEEE S&P*, 2014.

[21] V. Kuznetsov, J. Kinder, S. Bucur, and G. Candea. Efficient State Merging in Symbolic Execution. In *PLDI*, 2012.

[22] S. Meier, B. Schmidt, C. Cremers, and D. Basin. The TAMARIN Prover for the Symbolic Analysis of Security Protocols. In *CAV*, 2013.

[23] R. Milner. *Communication and Concurrency*. Prentice Hall, 1989.

[24] B. Möller, T. Duong, and K. Kotowicz. This POODLE Bites: Exploiting the SSL 3.0 Fallback. Google Security Advisory, 2014.

[25] G. Nelson and D. C. Oppen. Simplification by Cooperating Decision Procedures. *ACM TOPLAS*, 1979.

[26] R. Paige and R. E. Tarjan. Three Partition Refinement Algorithms. *SIAM Journal on Computing*, 1987.

[27] J. Somorovsky. Systematic Fuzzing and Testing of TLS Libraries. In *ACM CCS*, 2016.

[28] M. Weiser. Program Slicing. *IEEE Transactions on Software Engineering*, 1984.

[29] C. Cremers. The Scyther Tool: Verification, Falsification, and Analysis of Security Protocols. In *CAV*, 2008.

[30] C. Cremers. Scyther – Semantics and Verification of Security Protocols. Ph.D. thesis, Eindhoven University of Technology, 2006.

[31] N. Kobeissi, G. Nicolas, and M. Tiwari. Verifpal: Cryptographic Protocol Analysis for the Real World. In *INDOCRYPT*, 2020.

[32] A. Armando et al. The AVISPA Tool for the Automated Validation of Internet Security Protocols and Applications. In *CAV*, 2005.

[33] D. Basin, S. Mödersheim, and L. Viganò. OFMC: A Symbolic Model Checker for Security Protocols. *International Journal of Information Security*, 4(3):181–208, 2005.

[34] D. Antonioli, N. O. Tippenhauer, and K. Rasmussen. The KNOB is Broken: Exploiting Low Entropy in the Encryption Key Negotiation of Bluetooth BR/EDR. In *USENIX Security*, 2019.

[35] M. Vanhoef and E. Ronen. Dragonblood: Analyzing the Dragonfly Handshake of WPA3 and EAP-pwd. In *IEEE S&P*, 2020.